Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

К защите допустить:

Заведующий кафедрой ИИТ

_____ Д. В. Шункевич

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
по дисциплине «Модели решения задач в интеллектуальных системах»
на тему:

Пользовательский интерфейс интеллектуального ассистента для игровой среды

БГУИР КП6 6-05-0611-03 022 ПЗ

Студент гр. 321703 Р. Д. Давыдов
Проверяющий Д. А. Сальников

Минск 2026

СОДЕРЖАНИЕ

Перечень условных обозначений 2
Введение 3
1 Анализ подходов к разработке интерфейса интеллектуальной системы 5
1.1 Характеристика предметной области 5
1.2 Участники взаимодействия и их задачи 6
1.3 Анализ существующих систем и интерфейсов 8
1.4 Анализ требований к интерфейсу 10
1.5 Вывод по разделу анализа 12
2 Проектирование интеллектуальной системы 13
2.1 Общая архитектура интеллектуальной системы 13
2.2 Архитектура интерфейсного слоя 13
2.3 Модели пользователей и сценарии использования 14
2.4 Проектирование пользовательского интерфейса 19
2.5 Проектирование программного интерфейса 19
2.6 Проектирование интеллектуальных механизмов интерфейса 21
2.7 Вывод по разделу проектирования 22
3 Реализация интеллектуальной системы 23
3.1 Выбор средств реализации 23
3.2 Реализация программного (или аппаратного) интерфейса . . 26
3.3 Тестирование интерфейса 27
3.4 Вывод по разделу реализации 29
Заключение 31
Список использованных источников 34

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

В курсовом проекте используются следующие условные обозначения:

LLM (от англ. large language model) — большая языковая модель.

HUD (от англ. heads-up display) — интерфейс, который выводит важную информацию прямо перед глазами пользователя, чтобы он не отвлекался от основного занятия.

GUI (от англ. graphical user interface) — тип пользовательского интерфейса, позволяющий взаимодействовать с программным обеспечением через визуальные знаки и графические индикаторы.

ВВЕДЕНИЕ

Современный этап развития информационных технологий характеризуется фундаментальным сдвигом парадигмы человеко-машинного взаимодействия. Традиционные графические пользовательские интерфейсы (GUI), основанные на визуальных метафорах и прямом манипулировании элементами управления, постепенно дополняются, а в ряде случаев и замещаются интерфейсами на естественном языке. Особую актуальность данная тенденция приобретает в сфере интерактивных виртуальных сред и компьютерных игр, где сложность игровых механик и объем поступающей информации часто превышают когнитивные способности пользователя по их быстрой обработке. Актуальность темы курсового проекта обусловлена необходимостью создания новых подходов к проектированию пользовательских интерфейсов, способных не просто отображать статические данные, но и интерпретировать контекст происходящего, предоставляя пользователю релевантную обратную связь в интуитивно понятной форме. Разработка интеллектуальных ассистентов, интегрированных непосредственно в процесс взаимодействия с программным обеспечением, позволяет существенно снизить порог вхождения для новых пользователей и обогатить опыт опытных операторов системы. В условиях роста популярности больших языковых моделей и технологий синтеза речи открываются возможности для создания адаптивных интерфейсов, имитирующих социальное взаимодействие и обладающих элементами эмоционального интеллекта. Анализ достижений в предметной области показывает, что существующие решения в сфере голосовых помощников (Siri, Google Assistant, Алиса) преимущественно ориентированы на выполнение атомарных команд и поиск информации по прямому запросу пользователя. Однако они обладают существенным ограничением — отсутствием глубокой интеграции с контекстом текущей деятельности пользователя на экране (так называемая проблема контекстной слепоты). В игровой индустрии интерфейсы традиционно представлены в виде HUD (Heads-Up Display) и текстовых чатов, которые являются пассивными средствами отображения информации. Попытки внедрения NPC (неигровых персонажей) с искусственным интеллектом предпринимаются, однако они часто ограничены заранее прописанными сценариями поведения и отсутствием гибкости в диалоге. Проблема создания мультимодального интерфейса, который объединял бы в себе событийную осведомленность, визуальную персонализацию и голосовое взаимодействие в реальном времени, остается открытой инженерной и дизайнерской задачей. В качестве предметной области для разработки выбрана среда Minecraft, представляющая собой сложную систему с открытым миром и множеством неструктурированных событий, что делает ее идеальным полигоном для тестирования адаптивных интерфейсов. Целью курсового про-

екта является проектирование и разработка интеллектуального мультимодального интерфейса ассистента для виртуальной среды, обеспечивающего контекстное аудиовизуальное сопровождение пользователя на основе анализа событийного потока в реальном времени с использованием больших языковых моделей. Для достижения поставленной цели в ходе выполнения работы необходимо решить следующие задачи:

– провести анализ предметной области и существующих аналогов интеллектуальных ассистентов в игровых и прикладных средах, выявить недостатки текущих интерфейсных решений и сформулировать требования к проектируемой системе.

– разработать архитектуру программно-аппаратного комплекса, включающую модули сбора данных из приложения, серверную часть для агрегации событий и принятия решений, а также клиентские интерфейсы управления и отображения.

– спроектировать и реализовать серверную логику обработки естественного языка, обеспечивающую фильтрацию событий, генерацию контекстно-зависимых реплик и управление эмоциональной окраской синтезируемой речи.

– разработать пользовательские интерфейсы системы: веб-панель для настройки параметров личности ассистента и мониторинга его работы, а также внутриигровой оверлей для визуализации аватара и субтитров.

– провести тестирование разработанной системы, оценить задержки передачи данных и реакции интерфейса, а также проанализировать удобство взаимодействия пользователя с разработанным интеллектуальным ассистентом.

1 АНАЛИЗ ПОДХОДОВ К РАЗРАБОТКЕ ИНТЕРФЕЙСА ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ

1.1 Характеристика предметной области

Предметной областью разрабатываемой системы является человеко-машинное взаимодействие в рамках сложных виртуальных сред и имитационных моделей. В частности, проект фокусируется на проблематике создания интеллектуальных пользовательских интерфейсов, основанных на обработке естественного языка и генеративном искусственном интеллекте[1].

Современная индустрия интерактивных развлечений и виртуальных симуляций демонстрирует устойчивую тенденцию к усложнению игровых механик и увеличению объемов информации, поступающей пользователю в режиме реального времени. Традиционные подходы к организации интерфейсов, базирующиеся на графических элементах, меню и статических текстовых подсказках, часто оказываются недостаточно эффективными для обеспечения иммерсивного опыта. Пользователь вынужден переключать внимание между игровым процессом и интерфейсными элементами, что повышает когнитивную нагрузку и нарушает состояние потока[2].

В качестве полигона для исследования и разработки выбрана среда Minecraft — песочница с открытым миром, которая характеризуется высокой степенью энтропии событий. В отличие от линейных приложений, действия пользователя здесь не предопределены скриптами, что создает поток неструктурированных данных, который проблематично интерпретировать классическими алгоритмическими методами. Большие языковые модели способны выступать в роли когнитивного модуля, который не просто реагирует на атомарные триггеры, но и формирует целостную картину происходящего, обладает памятью и способен к ролевому моделированию[3].

Разрабатываемая интеллектуальная система ассистирования предназначена для решения комплекса задач, направленных на улучшение пользовательского опыта[4]:

– агрегация и интерпретация событийного потока, подразумевающая перехват низкоуровневых данных от клиента приложения и их преобразование в семантически значимые описания ситуаций;

– контекстно-зависимая генерация речи, требующая реализации механизма принятия решений о целесообразности речевого вмешательства вместо стандартной схемы запрос-ответ;

– эмоциональная адаптация, при которой ассистент поддерживает заданный характер, изменяя тональность ответов и стиль речи в зависимости от успеха или неудач пользователя;

– мультимодальное представление информации через синхронизированный вывод данных посредством аудиоканала и визуального оверлея.

Внедрение интеллектуального ассистента в виртуальную среду должно привести к следующим результатам:

– снижение когнитивной нагрузки, так как пользователь получает необходимую информацию через естественную речь, не отвлекаясь на чтение текстовых логов;

– повышение иммерсивности за счет замены механических интерфейсных уведомлений на диалог с персонифицированным персонажем;

– обеспечение доступности через дублирование информации по разным каналам восприятия.

Проектирование систем на базе языковых моделей в контексте реального времени накладывает ряд жестких технических и логических ограничений[5]:

– требования по времени отклика, согласно которым задержка между игровым событием и реакцией ассистента не должна превышать 1.5–2 секунды для сохранения иллюзии живого общения;

– ограничение контекстного окна языковой модели, требующее применения механизмов буферизации и суммаризации контекста для выделения только релевантных фактов из длительной сессии;

– склонность моделей к генерации правдоподобной, но ложной информации (галлюцинациям), что минимизируется через системные промпты и валидацию данных;

– большие объемы неструктурированных данных, генерируемых игровым клиентом, которые делают экономически и технически нецелесообразной прямую передачу всех событий в модель без предварительной фильтрации;

– этические аспекты, связанные с необходимостью фильтрации контента при имитации антагонистических характеров.

1.2 Участники взаимодействия и их задачи

Архитектура разрабатываемого программно-аппаратного комплекса предполагает взаимодействие между человеком-оператором, локальными программными компонентами и удаленными облачными сервисами. Участников информационного обмена можно классифицировать на четыре ключевые группы: конечный пользователь, внешняя виртуальная среда, внутренние подсистемы обработки данных и внешние когнитивные сервисы.

Конечный пользователь (оператор системы)

Основным выгодоприобретателем является пользователь, осуществляющий взаимодействие с виртуальной средой. Его роль заключается в генерации событийного потока через игровые механики и потреблении аудио-

визуальной обратной связи от ассистента. Задачи пользователя в системе:

– осуществление игровой деятельности, инициирующей поток событий (перемещение, взаимодействие с объектами, бой, строительство);

– настройка параметров личности ассистента и системных промптов через веб-интерфейс для персонализации опыта;

– восприятие синтезированной речи и визуальных индикаторов оверлея для корректировки своих действий в виртуальной среде.

Контекст использования предполагает наличие стационарного персонального компьютера, оснащенного устройствами вывода звука (наушники или акустическая система) и, опционально, вторым монитором для отображения панели управления.

Клиентское приложение виртуальной среды (Minecraft)

Внешняя система, выступающая генератором первичных данных. Роль данного участника — выполнение симуляции виртуального мира и фиксация изменений состояния. Задачи компонента:

– перехват внутриигровых триггеров посредством модификации кода игры;

– сериализация событий в структурированный формат (JSON) с добавлением метаданных (временная метка, координаты, тип события);

– передача данных на сервер обработки в режиме реального времени;

– исполнение команд обратной связи, поступающих от интеллектуальной системы (например, изменение погоды или выдача предметов).

Серверная часть системы (ядро обработки)

Центральный узел, оркестрирующий взаимодействие всех компонентов. Его цель — преобразование низкоуровневых сигналов игры в семантически значимый контент. Задачи подсистемы:

– агрегация и буферизация входящих событий для предотвращения избыточной частоты запросов;

– управление контекстом диалога и краткосрочной памятью ассистента;

– принятие решений о необходимости реакции на основе анализа текущей ситуации;

– маршрутизация запросов к внешним API и передача готового аудиовизуального контента на клиентские интерфейсы (оверлей и веб-панель).

Внешние сервисы искусственного интеллекта

Сторонние облачные или локальные решения, обеспечивающие когнитивные функции системы. К ним относятся большие языковые модели (LLM) и сервисы синтеза речи (TTS). Задачи сервисов:

– генерация естественного текста ответа на основе переданного системного промпта и контекста событий;

– классификация намерений пользователя и эмоциональная окраска реплик;

– преобразование текстовых строк в аудиопоток с заданными характеристиками тембра и скорости речи.

Взаимодействие между участниками осуществляется асинхронно, что позволяет поддерживать высокую отзывчивость интерфейса даже при задержках в обработке данных на стороне внешних сервисов.

1.3 Анализ существующих систем и интерфейсов

Проектирование нового интеллектуального интерфейса требует детального рассмотрения существующих решений в области автоматизации взаимодействия с виртуальными средами. На текущем этапе развития технологий можно выделить несколько классов систем, частично реализующих функционал ассистирования: императивные навигационные боты, исследовательские автономные агенты на базе больших языковых моделей и встроенные системы доступности.

Обзор аналогичных систем

В качестве объектов анализа выбраны три репрезентативные системы, демонстрирующие различные подходы к организации интерфейса человек-компьютер в контексте виртуальной среды Minecraft:

1. **Baritone (Pathfinding System).** Это наиболее распространенная система автоматизированной навигации и выполнения рутинных операций. Назначение системы заключается в алгоритмическом расчете оптимальных маршрутов и автоматическом выполнении действий (добыча ресурсов, строительство) без прямого управления персонажем. Тип интерфейса можно охарактеризовать как командно-текстовый (CLI over GUI), где взаимодействие происходит посредством ввода специальных директив в стандартный чат игры[6].

2. **Voyager (Embodied Agent).** Экспериментальная система, разработанная исследователями из NVIDIA и Caltech. Это первый воплощенный агент на базе LLM (GPT-4), способный к непрерывному обучению без участия человека. Назначение системы — автономное исследование мира, открытие новых технологий и накопление библиотеки навыков в виде исполняемого кода. Интерфейс системы относится к типу система-система, так как управление осуществляется программно, а роль человека сводится к наблюдению за логами и метриками через консольный вывод, без возможности прямого диалогового вмешательства в реальном времени[4].

3. **Стандартный экранный диктор (Minecraft Narrator).** Встроенный инструмент обеспечения доступности, преобразующий текстовые сообщения чата в речь. Назначение системы — обеспечение инклюзивности для пользователей с нарушениями зрения. Тип интерфейса — одно-

сторонний аудио-вывод, жестко привязанный к системным событиям графического интерфейса[7].

Анализ интерфейсов существующих решений

Детальный анализ пользовательского взаимодействия (UX) с перечисленными системами позволяет выявить фундаментальные ограничения текущих подходов и сформулировать требования к проектируемому мультимодальному ассистенту.

Анализ интерфейса системы Baritone

Ключевой сценарий взаимодействия с Baritone строится на использовании префиксов (обычно символа #) в игровом чате. Пользователь должен помнить синтаксис команд, например, директива `#mine diamond_ore` инициирует поиск алмазов. Структура интерфейса представляет собой гибрид графического отображения игрового мира и текстовой командной строки. Визуализация намерений бота реализуется через рендеринг цветных линий маршрута поверх изображения.

Сильные стороны решения заключаются в высокой детерминированности и точности исполнения: пользователь получает полный контроль над действиями аватара. Оверлей с траекторией движения обеспечивает прозрачность алгоритмических решений системы. Одновременно необходимость постоянно удерживать в памяти синтаксис специализированных директив и соотносить их с быстро меняющейся игровой ситуацией приводит к росту внешней когнитивной нагрузки на пользователя[8].

Однако проявляются критические ошибки проектирования интерфейса для массового пользователя:

– высокий порог вхождения, обусловленный необходимостью запоминания сотен команд и их параметров, что создает чрезмерную когнитивную нагрузку;

– отсутствие контекстной гибкости, так как система не понимает абстрактных запросов на естественном языке (например, «найди безопасное место») и требует четкой императивной формулировки;

– модальность ввода блокирует управление персонажем, поскольку для ввода команды необходимо открыть окно чата, что делает невозможным оперативное реагирование в критических ситуациях (например, во время боя).

Анализ интерфейса агента Voyager

Взаимодействие с системой Voyager происходит преимущественно в режиме наблюдателя. Пользователь запускает скрипт инициализации, после чего система работает автономно, генерируя код навыков на языке JavaScript. Структура интерфейса для оператора представлена набором текстовых логов в терминале и визуальным наблюдением за действиями бота в окне игры.

Сильной стороной является полная автоматизация когнитивных про-

цессов: система сама ставит цели и достигает их. Это демонстрирует потенциал использования LLM для сложного планирования, долговременного накопления контекста и построения поведенческих циклов, приближающих агента к модели интерактивного цифрового персонажа[9].

С точки зрения пользовательского интерфейса ассистента, данное решение имеет существенные недостатки:

– отсутствие интерактивности, так как архитектура Voyager не предусматривает канала обратной связи от пользователя к агенту в реальном времени;

– непрозрачность принятия решений для неквалифицированного пользователя, который видит лишь конечный результат действий, но не понимает мотивацию агента без анализа программного кода;

– задержка реакции, обусловленная итеративным процессом генерации и отладки кода, что неприемлемо для задач оперативного ассистирования.

Анализ интерфейса экранного диктора (Narrator)

Сценарий использования сводится к пассивному прослушиванию синтезированной речи, дублирующей текстовые сообщения других игроков и системные уведомления. Интерфейс не имеет визуального представления, управление осуществляется через меню настроек.

Преимуществом является простота интеграции и низкое потребление ресурсов. Однако функциональность системы ограничена прямым чтением текста без семантического анализа.

Слабые стороны включают:

– отсутствие фильтрации контекста, приводящее к зашумлению аудиоканала (информационный спам), когда диктор озвучивает технические логи или повторяющиеся сообщения;

– монотонность и отсутствие эмоциональной окраски, что снижает вовлеченность и не позволяет использовать систему как полноценного компаньона;

– отсутствие мультимодальности, так как аудиопоток не подкрепляется визуальными образами или анимацией, что затрудняет восприятие информации в шумной обстановке.

1.4 Анализ требований к интерфейсу

На основе исследования предметной области и ролевой модели участников взаимодействия определены требования к интерфейсу интеллектуальной системы ассистирования.

Функциональные требования

Функциональность интерфейса распределена между конечным пользователем (игроком) и администратором системы:

– интерфейс игрового клиента (мод) должен обеспечивать автоматический сбор и передачу структурированных данных о состоянии аватара (HP, уровень сытости) и окружающей среды (наличие мобов, время суток) на сервер.

– интерфейс обратной связи (внутри игры) должен обеспечивать визуальное отображение текстовых советов и предупреждений, полученных от сервера, в удобной для восприятия форме (чат или графический оверлей).

– веб-интерфейс администратора должен обеспечивать визуализацию потока игровых событий в режиме реального времени посредством протокола WebSocket для оперативного мониторинга состояния системы.

– интерфейс управления конфигурацией должен обеспечивать возможность изменения рабочих параметров сервера (например, *analysis_interval*) через веб-форму с немедленной синхронизацией настроек с игровым клиентом.

– система должен обеспечивать автоматическую валидацию входящих сообщений на соответствие схеме данных *LogEntry* перед их отображением или анализом.

Нефункциональные требования

Для обеспечения стабильной работы в условиях высокодинамичной среды Minecraft к системе предъявляются следующие требования:

– надежность: для предотвращения сбоев при передаче данных необходимо использовать строгую валидацию типов через *Pydantic*. Не допускается потеря событий при кратковременных разрывах соединения (необходима буферизация на стороне клиента).

– удобство и доступность: интерфейс админ-панели следует реализовывать на базе стандартных веб-технологий (HTML5/JS), обеспечивающих доступность через любой современный браузер без установки дополнительного ПО.

– ограничения среды: клиентская часть должна быть совместима со средой *Fabric 1.20.1* и *Java 17*[10]. Не допускается блокировка основного игрового потока (Render Thread) при выполнении сетевых запросов — все операции обмена данными должны быть асинхронными.

– расширяемость: архитектуру системы следует проектировать таким образом, чтобы замена *rule-based* логики на *LLM-модуль* не требовала переработки интерфейсов взаимодействия.

Требования к «интеллектуальности» интерфейса

Интеллектуальная составляющая интерфейса в рамках курсового проекта реализуется через событийный анализ и адаптивную подачу информации:

– адаптивная фильтрация (Троттлинг): интерфейс должен реализовывать логику подавления избыточных уведомлений. Не допускается по-

вторная выдача одного и того же совета (например, о наступлении ночи) чаще, чем установлено заданным лимитом времени (от 10 до 60 секунд в зависимости от типа события).

– приоритизация контекста: при возникновении нескольких триггеров одновременно (например, голод и низкое здоровье) система должна выбирать приоритетный совет. Логика выбора должна базироваться на степени угрозы для выживания игрового персонажа.

– интеллектуальная обработка состояний: система не просто дублирует игровые логи, а интерпретирует их. Например, на основе типа события *near_hostile* и данных о расстоянии интерфейс должен формировать предиктивное предупреждение об угрозе атаки.

– уровень сложности: для текущего прототипа устанавливается уровень *экспертной системы на базе правил (rule-based)*. Интеллектуальность подтверждается наличием четких алгоритмов обработки контекста, описанных в коде сервера (*advice.py*), а не случайной выдачей фраз.

1.5 Вывод по разделу анализа

На основе проведенного анализа можно сделать следующие выводы:

– предметная область характеризуется высокой энтропией данных, что требует реализации посредника между сырыми событиями игры и пользователем для снижения когнитивной нагрузки.

– анализ существующих решений показал, что большинство аналогов либо излишне сложны (Baritone), либо не обеспечивают интерактивного взаимодействия в реальном времени (Voyager), что подтверждает актуальность разработки легковесного ассистента.

– в качестве архитектурной основы выбрана клиент-серверная модель: клиент на базе *Fabric* для сбора данных и сервер на *FastAPI* для анализа контекста и управления настройками.

– принято решение использовать *rule-based* подход для генерации советов на этапе MVP, что обеспечит детерминированность поведения системы и заложит фундамент для последующей интеграции языковых моделей.

2 ПРОЕКТИРОВАНИЕ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ

2.1 Общая архитектура интеллектуальной системы

Интеллектуальная система ассистирования построена по трёхуровневой клиент-серверной архитектуре. На нижнем уровне находится клиентский модуль — модификация Minecraft на платформе Fabric, задача которого заключается исключительно в сборе и передаче событий. На среднем уровне расположено серверное ядро на базе *FastAPI*, выполняющее агрегацию событий, управление диалоговым контекстом и принятие решений о генерации реплик. Верхний уровень представлен внешними когнитивными сервисами (Groq API, Ollama), к которым сервер обращается для генерации естественноязыкового текста.

Подсистемы системы и их связи:

– Minecraft-мод (клиент) — перехватчик событий, сериализатор JSON-сообщений, исполнитель команд от сервера, визуальный оверлей;

– серверное ядро (FastAPI) — маршрутизатор HTTP/WebSocket запросов, хранилище событий (*logs_db*), менеджер настроек, оркестратор LLM-вызовов;

– модуль LLM-клиента (*groq_client.py*) — абстракция над внешними API, формирователь промптов, резервный rule-based генератор;

– веб-панель администратора — браузерный интерфейс для мониторинга событий, истории ответов и конфигурирования системы;

– внешние LLM-сервисы — Groq (облачный) и Ollama (локальный) для генерации текста.

2.2 Архитектура интерфейсного слоя

Структура модулей и слоёв

Серверная часть системы декомпозирована на четыре функциональных модуля, каждый из которых обслуживает отдельный аспект интерфейса:

– `api/logs.py` — приёмный буфер событий. Модуль предоставляет конечную точку `POST /api/logs/` для регистрации событий от мода и транслирует их в режиме реального времени через WebSocket (`/api/logs/ws`) подключённым клиентам панели мониторинга. Данные хранятся во временном списке *logs_db* (максимум 10 000 записей) с автоматическим вытеснением устаревших записей;

– `api/settings.py` — менеджер конфигурации. Реализует CRUD-интерфейс `GET/PUT /api/settings/` для объекта настроек *Settings* (*analysis_interval*, *threat_threshold*, *enable_voice*, *max_threats_display*). На-

стройки персистируются в файле `settings.json`;

– `api/advice.py` — оркестратор интеллектуальных реакций. Это центральный модуль системы, обрабатывающий запросы на генерацию ролевых реплик (`GET /api/rp/`), ответов на сообщения игрока в чате (`POST /api/rp/chat`), а также управляющий переключением моделей (`POST /api/rp/models/switch`). Модуль ведёт историю ответов и рассылает их в реальном времени через WebSocket (`/api/rp/ws`);

– `api/groq_client.py` — адаптер LLM-провайдеров. Инкапсулирует логику обращения к Groq и Ollama, формирует промпты из шаблонов конфигурации, обеспечивает асинхронный вызов блокирующих API через пул потоков (*ThreadPoolExecutor*).

Конфигурация активной модели вынесена в файл `llm_config.json`, что позволяет администратору переключаться между профилями (*llama*, *qwen*, *qwen3-local*) без перезапуска сервера. Загрузка и валидация конфигурации выполняются модулем `api/llm_config_manager.py` посредством *Pydantic*-схемы *LLMConfig*.

Взаимодействие с внутренними компонентами

Взаимодействие между модулями сервера осуществляется через общую in-memory структуру *logs_ db* (список объектов *LogEntry*). Модуль `advice.py` читает из неё последние N записей при каждом запросе генерации реплики.

Все исходящие сетевые операции (вызовы Groq/Ollama) выполняются асинхронно: блокирующие функции `generate_tip()` и `generate_chat_response()` оборачиваются в `asyncio.run_in_executor()`, что не допускает блокировки основного event-loop FastAPI.

Механизм обработки ошибок реализован по принципу деградации: при недоступности LLM-сервиса (отсутствие API-ключа, сетевая ошибка) управление передаётся rule-based генератору `_get_fallback_rp_response()`, который формирует детерминированный ответ на основе анализа типов событий в *logs_ db*.

2.3 Модели пользователей и сценарии использования

Ролевые модели пользователей и внешних систем

С учётом разработанной архитектуры уточнены следующие роли участников:

– игрок — пассивный получатель информации от ассистента внутри игры. Не имеет прямого доступа к серверу; взаимодействует через мод (перехват событий, отображение оверлея, сообщения в чате). Ограничен сценариями виртуальной среды;

– администратор — оператор, управляющий системой через вебпанель. Имеет доступ к просмотру журнала событий, истории ответов

ИИ, изменению настроек (*threat_threshold*, *analysis_interval*), переключению модели LLM. Как правило, совпадает с игроком в однопользовательском сценарии;

– Minecraft-мод — автоматизированная внешняя система-источник данных. Не имеет собственного пользовательского интерфейса; взаимодействует с сервером посредством HTTP-запросов;

– LLM-сервис — внешний когнитивный провайдер (Groq или Ollama). Получает структурированный промпт, возвращает текст реплики.

Сценарии использования

Выделены три основных сценария взаимодействия с интерфейсом системы.

Сценарий 1: Автоматическая контекстная реакция ассистента. (рисунок 2.1) Мод периодически опрашивает сервер по адресу `GET /api/rp/`. Сервер извлекает последние 5 записей *logs_db*, формирует контекстный промпт и обращается к LLM. Полученная реплика проходит проверку частотного ограничителя (*rate limiter*): если с момента последней реплики прошло менее *_min_interval_seconds* (20 с по умолчанию) и ситуация не изменилась, возвращается кэшированный ответ. Готовая реплика возвращается моду и одновременно рассылается через WebSocket подключённым браузерным клиентам.

Альтернативная ветвь: если событие принадлежит к классу критических (*CRITICAL_EVENTS* = *{"death"}*), ограничитель обходится и реплика генерируется немедленно.

Ошибочная ветвь: при недоступности LLM возвращается ответ rule-based генератора с пометкой `source: "fallback"`.

Сценарий 2: Диалоговое взаимодействие (чат). (рисунок 2.2) Игрок вводит сообщение в игровой чат. Мод перехватывает его и отправляет на `POST /api/rp/chat` с полем `text`. Сервер формирует chat-промпт из шаблона *chat_based* (передаётся сообщение игрока и последние 5 событий контекста) и запрашивает LLM. Ответ возвращается моду и записывается в историю с пометкой `source: "chat"`.

Сценарий 3: Мониторинг и настройка. (рисунок 2.3) Администратор открывает веб-панель в браузере. На странице мониторинга (`/admin`) устанавливается WebSocket-соединение с `/api/logs/ws`: новые события отображаются в реальном времени. На странице истории ответов (`/admin/responses`) аналогичным образом отображаются реплики ИИ через `/api/rp/ws`. На странице настроек (`/admin/settings`) администратор через HTML-форму отправляет `PUT /api/settings/` с новыми параметрами; там же переключает активную модель через `POST /api/rp/models/switch`.

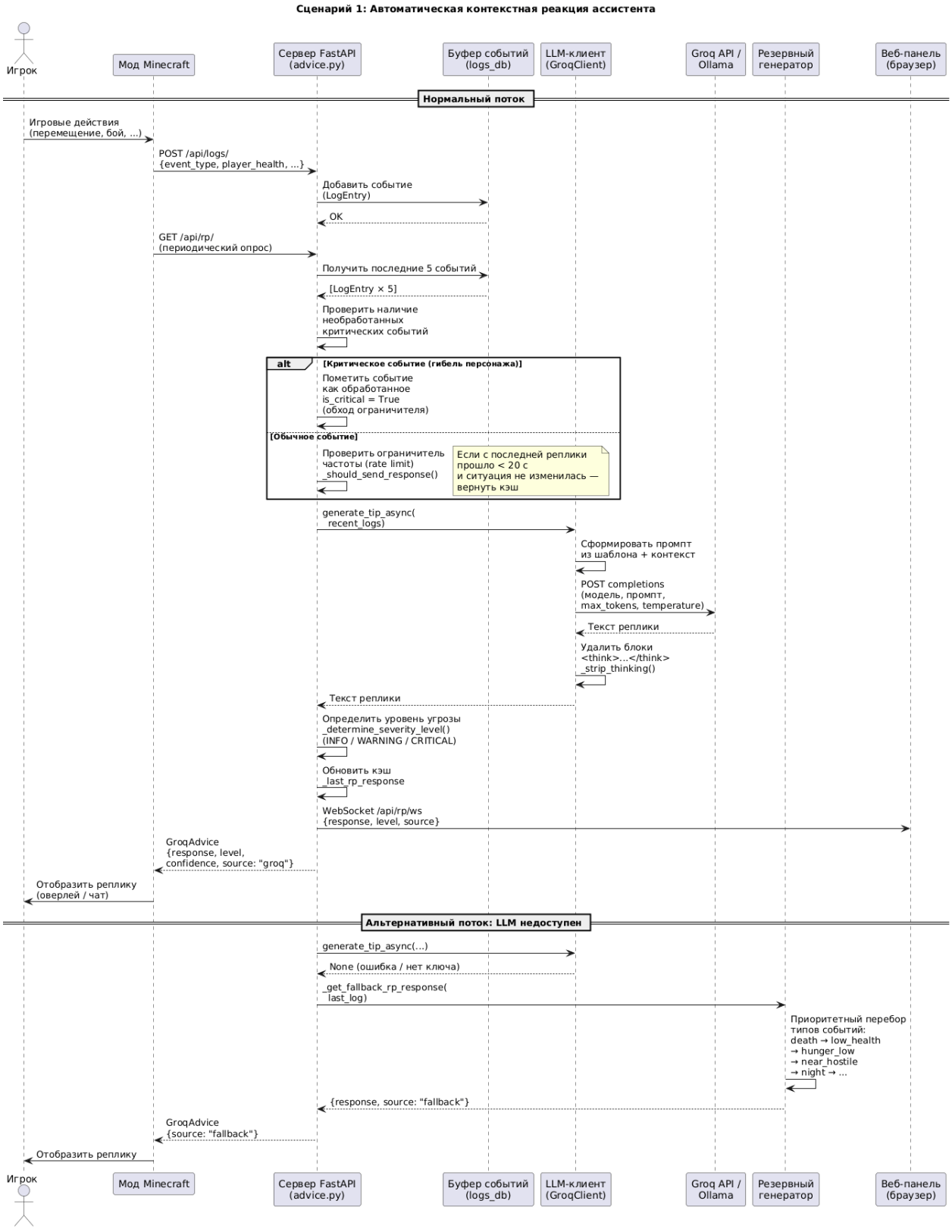


Рисунок 2.1 – Диаграмма последовательности для сценария 1

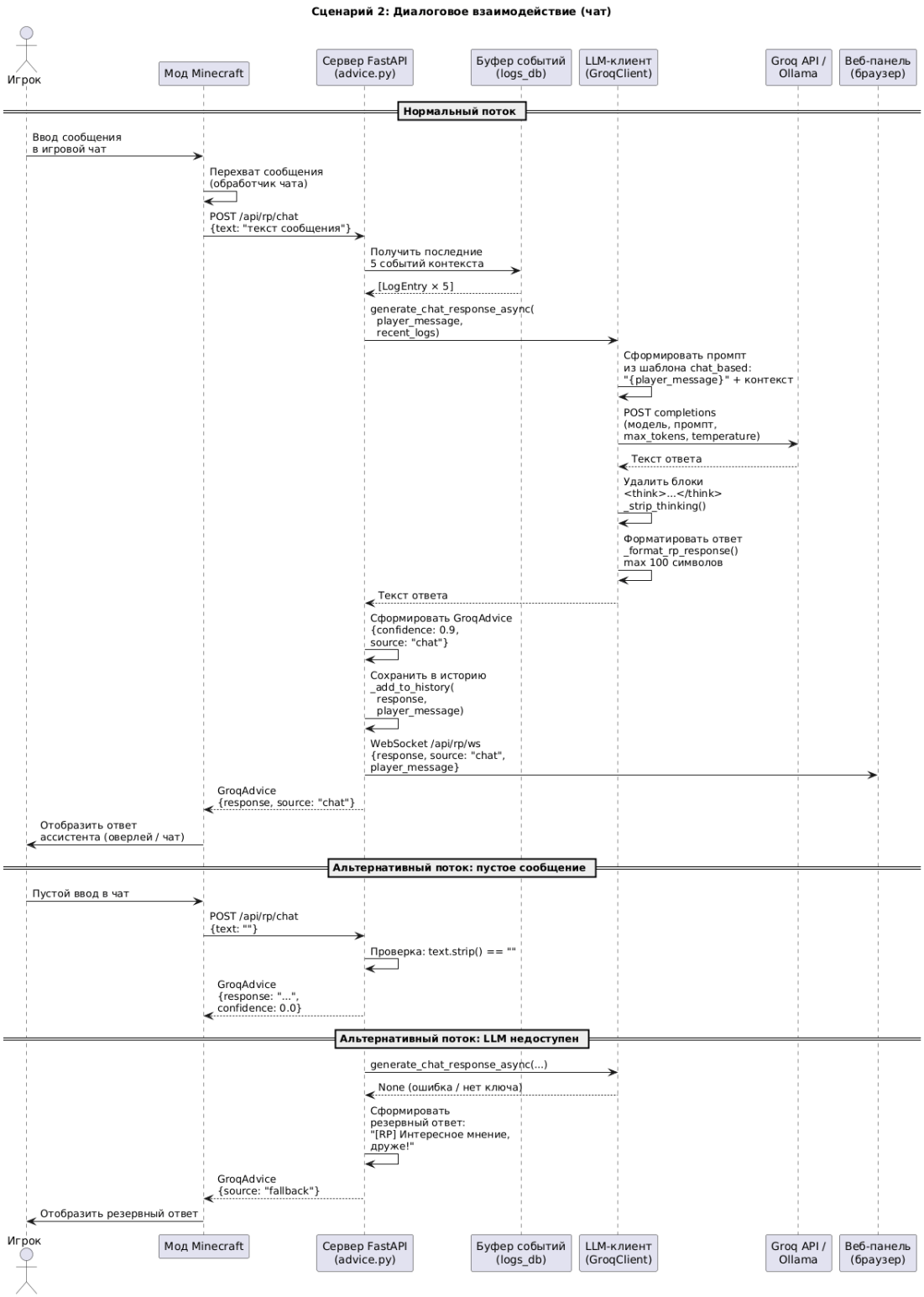


Рисунок 2.2 – Диаграмма последовательности для сценария 2

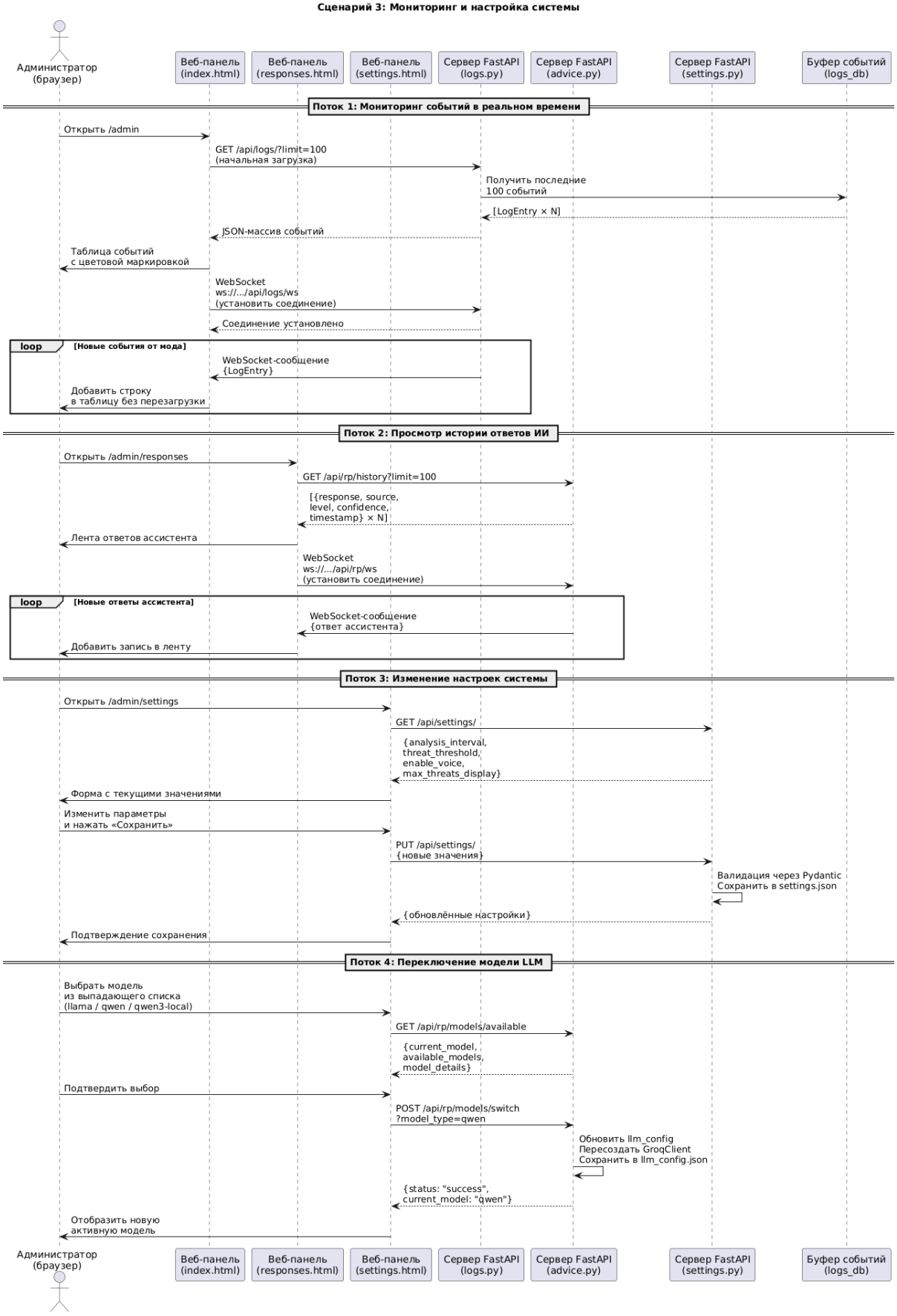


Рисунок 2.3 – Диаграмма последовательности для сценария 3

2.4 Проектирование пользовательского интерфейса

Информационная структура и навигация

Браузерная административная панель состоит из трёх страниц с независимой навигацией:

– мониторинг событий (`/admin`, файл `index.html`) — основная страница. Отображает поток игровых событий в виде таблицы с колонками: временная метка, уровень важности (*INFO*/*WARNING*/*ERROR*), тип события, данные события, здоровье игрока;

– история ответов ИИ (`/admin/responses`, файл `responses.html`) — лента реплик ассистента с указанием источника (*groq*/*fallback*/*chat*), уверенности и временной метки;

– настройки (`/admin/settings`, файл `settings.html`) — форма редактирования параметров системы и переключения активной LLM-модели.

Переход между страницами осуществляется через навигационные ссылки в заголовке панели; каждая страница является самостоятельным HTML-документом, подключающим общую таблицу стилей `style.css`.

Макеты экранов и элементы управления

На странице мониторинга событий размещены: кнопка очистки журнала, переключатель автопрокрутки и таблица с цветовой маркировкой строк по уровню важности (зелёный — INFO, жёлтый — WARNING, красный — ERROR). Обновление происходит без перезагрузки страницы благодаря WebSocket.

На странице настроек расположены числовые поля ввода для *analysis_ interval* (мс) и *threat_ threshold* (0.0–1.0), чекбокс включения голосового вывода (*enable_ voice*), поле целого числа для *max_ threats_ display*, а также выпадающий список доступных LLM-моделей с кнопкой применения. Все элементы снабжены подписями и подсказками допустимых диапазонов.

2.5 Проектирование программного интерфейса

Спецификация операций и сообщений

Серверное API состоит из следующих конечных точек:

– `POST /api/logs/` — регистрация игрового события. Тело запроса: объект *LogEntry* (JSON). Обязательные поля: `level` (строка *INFO*/*WARNING*/*ERROR*). Необязательные: `event_type`, `event_data` (словарь), `player_health` (число), `threats_detected` (массив). Ответ: `{"status": "added"}` с кодом 200;

– `GET /api/logs/` — получение последних N событий. Параметры запроса: `limit` (целое, по умолчанию 100), `level` (строка, необязательно). Ответ: массив объектов *LogEntry*;

– `GET /api/rp/` — получение контекстной реплики ассистента. Тело запроса отсутствует. Ответ: объект *GroqAdvice* (`response`, `confidence`, `level`, `threats`, `source`);

– `POST /api/rp/chat` — ответ на сообщение игрока в чате. Тело: объект *ChatMessage* (`text`: строка). Ответ: *GroqAdvice*;

– `GET /api/settings/` — получение текущих настроек. Ответ: объект *Settings*;

– `PUT /api/settings/` — обновление настроек. Тело: объект *Settings*. Ответ: обновлённый объект *Settings*;

– `GET /api/rp/models/available` — список доступных LLM-моделей с метаданными. Ответ: `{"current_model "available_-models "model_details"}`;

– `POST /api/rp/models/switch?model_type={name}` — переключение активной модели. Ответ: `{"status "current_model "model_info"}`.

Для потоковой передачи данных в реальном времени предусмотрены два WebSocket-соединения:

– `ws://.../api/logs/ws` — трансляция новых событий *LogEntry*;

– `ws://.../api/rp/ws` — трансляция новых ответов ассистента.

Контракты и протоколы взаимодействия

Все HTTP-эндпоинты функционируют по протоколу REST поверх HTTP/1.1. Формат данных — JSON с кодировкой UTF-8. Схемы запросов и ответов определены через *Pydantic*-модели; нарушение схемы возвращает код 422 Unprocessable Entity с детализацией ошибок валидации.

WebSocket-соединения используют субпротокол по умолчанию; каждое сообщение представляет собой JSON-сериализованный объект соответствующего типа.

Ключевые схемы данных:

– *LogEntry*: `timestamp` (ISO-8601), `level` (строка), `message` (строка, опционально), `event_type` (строка, опционально), `event_data` (объект, опционально), `player_health` (число 0–20, опционально), `threats_detected` (массив, опционально);

– *Settings*: `analysis_interval` (целое, мс, по умолчанию 500), `threat_threshold` (число 0.0–1.0, по умолчанию 0.7), `enable_voice` (булево), `max_threats_display` (целое);

– *GroqAdvice*: `response` (строка, максимум 100 символов после форматирования), `confidence` (число 0.0–1.0), `level` (*INFO*/*WARNING*/*CRITICAL*), `threats` (массив), `source` (*groq*/*fallback*/*chat*).

2.6 Проектирование интеллектуальных механизмов интерфейса

Интеллектуальная поддержка пользователя

Интеллектуальный механизм формирования реплик функционирует в двух режимах, между которыми осуществляется автоматическое переключение:

Основной режим — LLM-генерация. При доступном API-ключе модуль *GroqClient* формирует промпт из двух составляющих: системного шаблона (хранится в *llm_config.json*, отдельно для событийного и чат-сценария) и контекстного блока из последних 5 событий *logs_db*. Параметры вызова (модель, температура, *max_tokens*) также берутся из конфигурации, что обеспечивает возможность оперативной настройки поведения модели без изменения кода.

Поддерживаются три профиля модели:

– llama — `llama-3.1-8b-instant` через Groq (облако, низкая задержка);

– qwen — `qwen/qwen3-32b` через Groq с параметром *reasoning_effort: none*;

– qwen3-local — `qwen3:8b` через Ollama (локальный вывод, без передачи данных во внешние сети).

Для моделей Qwen, генерирующих расширенные блоки рассуждения в тегах `<think>...</think>`, предусмотрена функция `_strip_thinking()`, удаляющая эти блоки из ответа регулярным выражением перед отправкой игроку.

Резервный режим — rule-based генерация. При недоступности LLM вызывается `_get_fallback_rp_response()`, реализующий приоритетную очередь обработки событий: *death* → *low_health* → *hunger_low* → *near_hostile* → *night* → анализ числового значения *player_health* → анализ *threat_score*. Каждой ветви соответствует заранее написанная реплика на русском языке с типовым уровнем уверенности (*confidence*).

Объяснимость и адаптация интерфейса

Механизм частотного ограничения (*rate limiting*) решает задачу предотвращения информационного перегруза. Реализован через глобальные переменные *_last_rp_response_time* и *_min_interval_seconds*. Проверка `_should_send_response()` возвращает `False`, если с момента последней реплики прошло менее 20 секунд, — в этом случае возвращается кэшированный ответ *_last_rp_response*. Критические события (гибель персонажа) обходят ограничитель через флаг *is_critical*.

Детектор изменения ситуации `_has_situation_changed()` сравнивает уровень серьёзности (*level*) и источник (*source*) текущего и предыдущего ответа, а также первые 20 символов текста реплики. Это предотвращает

отправку дублирующихся уведомлений при стабильном состоянии игрока.

Объяснимость ответов обеспечивается полем `source` в структуре *GroqAdvice*: значение *groq* сигнализирует об использовании языковой модели, *fallback* — о применении детерминированных правил. Поле `level` (*INFO* / *WARNING* / *CRITICAL*) отражает степень угрозы, вычисленную функцией `_determine_severity_level()` на основе *player_health* и *threat_score*.

Адаптация к контексту реализована через конфигурируемый порог угрозы *threat_threshold* (по умолчанию 0.7): значения выше порога переводят систему в режим *WARNING*, что меняет тональность генерируемых реплик.

2.7 Вывод по разделу проектирования

В ходе проектирования приняты следующие ключевые архитектурные решения:

– трёхуровневая клиент-серверная архитектура с чётким разделением ответственности: мод собирает данные, сервер принимает решения, LLM генерирует текст. Это обеспечивает выполнение требования расширяемости: замена rule-based логики на LLM не затрагивает интерфейс мода;

– асинхронная обработка LLM-вызовов через *ThreadPoolExecutor* не блокирует основной поток FastAPI и позволяет обслуживать WebSocket-соединения с нулевой дополнительной задержкой для браузерных клиентов;

– двухуровневая деградация (LLM → rule-based fallback) обеспечивает непрерывность работы ассистента при отказе внешних сервисов, выполняя требование надёжности из раздела анализа;

– конфигурируемые профили LLM в `llm_config.json` позволяют оперативно переключаться между моделями (облачные Groq, локальный Ollama) без изменения кода, что реализует требование к независимости от конкретного провайдера;

– механизм rate limiting и детектор изменения ситуации реализуют требование к адаптивной фильтрации избыточных уведомлений, обеспечивая оптимальную частоту обратной связи;

– строгая типизация через Pydantic (*LogEntry*, *Settings*, *GroqAdvice*) гарантирует корректность данных на границах всех подсистем, выполняя требование надёжности передачи событий.

Предусмотренные возможности расширения: добавление новых типов событий через расширение *LogEntry.event_type* без изменения серверной логики; подключение новых LLM-провайдеров через добавление профиля в `llm_config.json`; интеграция модуля синтеза речи (TTS) через флаг *enable_voice* в *Settings* без изменения API мода.

3 РЕАЛИЗАЦИЯ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ

3.1 Выбор средств реализации

Языки программирования, фреймворки и библиотеки

Практическая реализация интеллектуальной системы выполнена как связка серверного приложения, клиентского мода и веб-интерфейса администратора. Для серверной части выбран язык Python 3.11, обеспечивающий высокую скорость прототипирования и достаточную выразительность для работы с сетевыми интерфейсами, JSON-структурами и интеграцией внешних API. В качестве веб-фреймворка используется *FastAPI*, поверх которого реализованы REST-эндпоинты, WebSocket-каналы и схемы валидации входящих и исходящих сообщений. Запуск приложения выполняется через *Uvicorn*, а строгая типизация данных обеспечивается библиотекой *Pydantic*. Для автоматизированного браузерного smoke-тестирования интерфейса дополнительно использована библиотека *Selenium*, позволяющая проверять реальные пользовательские сценарии в headless-браузере.

Клиентская часть реализована на языке Java с использованием платформы модификации *Fabric* для Minecraft 1.20.1. Данный выбор обусловлен необходимостью встраивания непосредственно в игровой клиент, перехвата событий игрового мира и исполнения команд обратной связи. Используемая конфигурация проекта ориентирована на Java 17, Fabric Loader 0.15.x и Fabric API 0.92.x, что соответствует актуальному набору инструментов для разработки клиентских модификаций под выбранную версию игры[10].

Веб-панель администратора построена на стандартных технологиях HTML, CSS и JavaScript без применения тяжёлых frontend-фреймворков. Такое решение позволяет минимизировать сложность сопровождения, сохранить прозрачность структуры интерфейса и быстро расширять состав административных страниц. Для интеграции языковых моделей на сервере используются клиенты Groq и OpenAI-compatible API для Ollama; конфигурация моделей вынесена в отдельные JSON-файлы `llm_config.json` и `llm_prompts.json`.

Обоснование выбора средств

Выбор Python и FastAPI обусловлен архитектурой проектируемой системы, в которой сервер выступает промежуточным интеллектуальным слоем между модом и внешними LLM-сервисами. Серверу требуется одновременно принимать частые телеметрические сообщения, обслуживать браузерные подключения в реальном времени, выполнять вызовы внешних API и не блокировать пользовательский сценарий. FastAPI предоставляет единый механизм описания REST-операций, WebSocket-взаимодействия и

схем данных, что снижает число потенциальных ошибок на границах модулей и упрощает расширение API.

Использование Fabric на стороне Minecraft-мода оправдано требованиями к глубокой интеграции с клиентом игры. Мод должен получать доступ к игровому чату, состоянию игрока, инвентарю, событиям гибели сущностей и клиентскому циклу обновления. Платформа Fabric предоставляет для этого необходимый набор точек расширения при сравнительно низком пороге модификации базового клиента. Дополнительным преимуществом является возможность разделить клиентские и серверные обязанности внутри самого мода: телеметрия и выполнение команд остаются в управляемом коде, а бизнес-логика сосредоточена на стороне FastAPI-сервера.

Отказ от frontend-сборщика и использование нативного JavaScript также является осознанным архитектурным решением. Административная панель в данном проекте не требует сложной компонентной модели, но должна быстро отображать поток событий, историю ответов и конфигурационные параметры. Структура из отдельных HTML-страниц и специализированных JS-файлов облегчает сопровождение, а также обеспечивает прямое соответствие между серверными маршрутами и пользовательскими экранами.

Реализация пользовательского интерфейса

Пользовательский интерфейс системы реализован в двух взаимодополняющих контурах: внутриигровом и веб-контуре. Внутриигровой контур обслуживается классом `AssistantRuntime` в клиентском моде. Он отвечает за периодический опрос серверных эндпоинтов, отправку пользовательских сообщений из игрового чата, получение командных реакций и отображение ответов ассистента в чате Minecraft. Таким образом, игрок взаимодействует с интеллектуальной системой без выхода из игрового сценария: телеметрия собирается автоматически, а ответы и команды выдаются в привычной для Minecraft форме (рисунок 3.1).



Рисунок 3.1 – Общение внутри игры

Веб-контур разделён на два уровня (рисунок 3.2). Первый уровень — классическая административная панель с маршрутами `/admin`, `/admin/responses`, `/admin/settings`, `/admin/analytics`,

/admin/player-state, /admin/challenges и /admin/wiki. Эти страницы предназначены для мониторинга, отладки и ручного управления конфигурацией системы. Второй уровень — более современный интерфейс верхнего уровня с маршрутами /dashboard и /steam, ориентированный уже не на таблицы и формы администрирования, а на визуальное представление состояния системы и работу с базой крафтов в более цельной пользовательской оболочке.

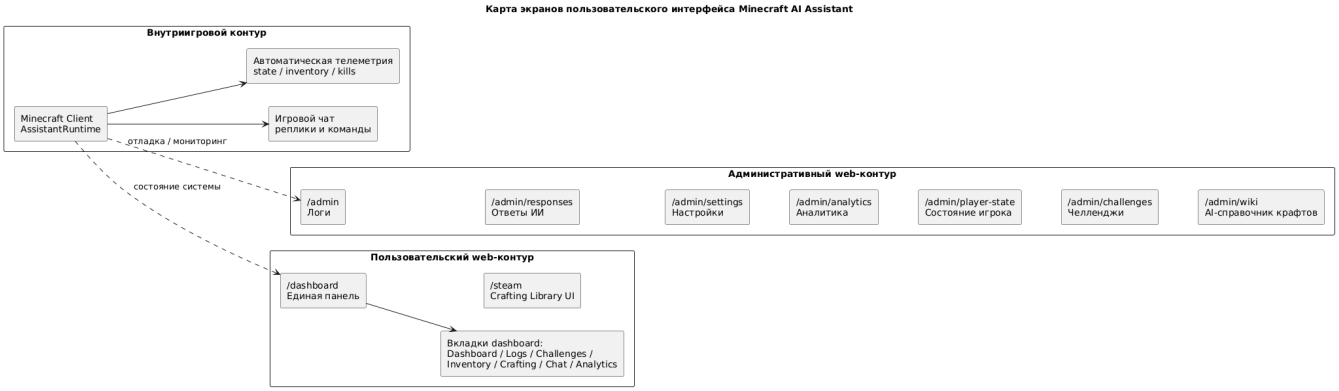


Рисунок 3.2 – Карта экранов

Страница /dashboard (рисунок 3.3) реализована как одностраничный интерфейс с верхней навигацией и боковой панелью. На ней объединены несколько функциональных экранов: сводный dashboard, live logs, управление челленджами, просмотр инвентаря, раздел крафтов, чат с ассистентом и аналитика игровой сессии. Переключение между экранами происходит на клиентской стороне без перезагрузки документа, а данные подгружаются через REST-запросы к /api/logs/, /api/player-state/, /api/challenges/, /api/recipes и другим маршрутам. Для ленты логов используется постоянное WebSocket-соединение с /api/logs/ws, благодаря чему изменения отображаются в реальном времени.

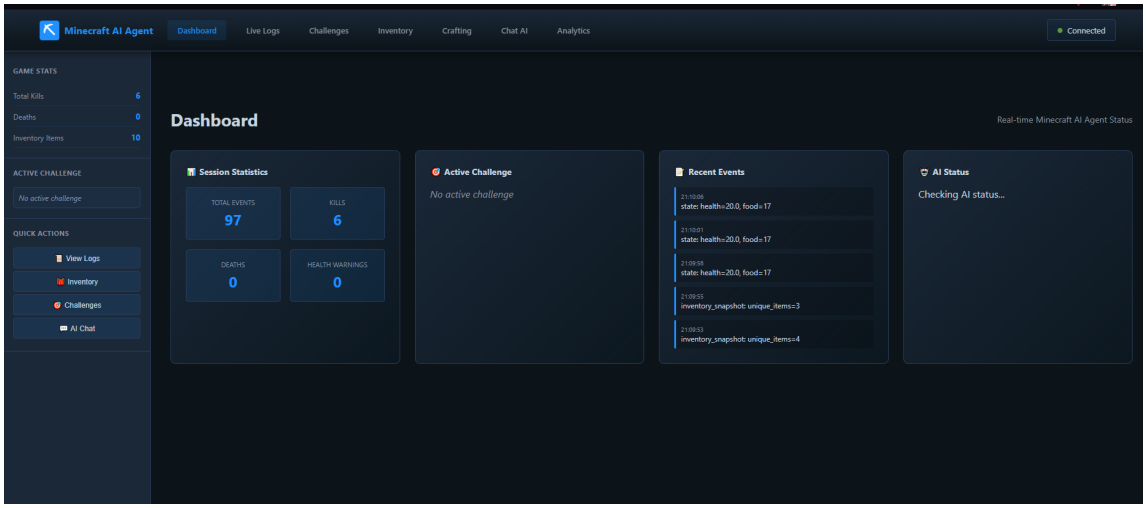


Рисунок 3.3 – Страница Dashboard

Страница `/steam` (рисунок 3.4) реализует альтернативный “витринный” интерфейс справочника крафтов в стилистике цифровой игровой библиотеки. Здесь предусмотрены домашний экран, библиотека рецептов с карточками, фильтрация по категориям, поиск, избранное и модальное окно просмотра конкретного рецепта. Рецепты подгружаются из локальной JSON-базы через эндпоинт `/api/recipes`, а логика клиента распределена между стилями `steam.css` и скриптом `steam.js`. Такой подход позволил отделить административные функции системы от более дружелюбного исследовательского пользовательского сценария.

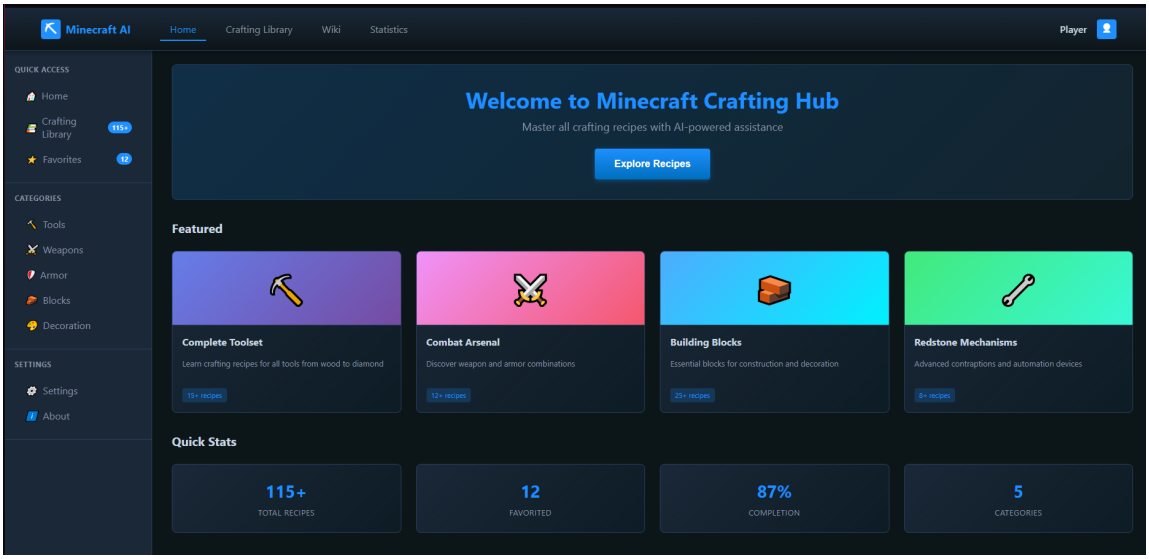


Рисунок 3.4 – Страница Dashboard

Отдельное место занимает обновлённая страница `/admin/wiki`, которая из простого текстового экрана превратилась в интерактивный AI-справочник крафтов. Пользователь вводит название предмета, после чего интерфейс запрашивает рецепт через `/api/wiki/craft` и визуализирует его в виде 3×3 сетки верстака с названием и пояснением результата. Поиск сначала опирается на локальную базу рецептов, а при отсутствии совпадения может использовать LLM как fallback. Для повышения читаемости пользовательских данных веб-интерфейс получает от сервера не только системные идентификаторы `minecraft:*`, но и локализованные русские названия сущностей и предметов.

3.2 Реализация программного (или аппаратного) интерфейса

Программный интерфейс системы реализован как совокупность REST-эндпоинтов и WebSocket-каналов. Основной поток телеметрии начинается на стороне мода: клиент формирует структурированные события

и отправляет их на `POST /api/logs/`. На сервере записи проходят валидацию по схеме *LogEntry*, сохраняются в общем буфере *logs_db*, а затем ретранслируются подписчикам через `/api/logs/ws`. Поверх тех же событий обновляется дополнительное состояние игрока: счётчики убийств, состояние инвентаря и история последних действий.

Модуль `api/advice.py` реализует программный интерфейс интеллектуального взаимодействия. Эндпоинт `GET /api/rp/` формирует контекстную реакцию на последние события, а `POST /api/rp/chat` обслуживает инициативные сообщения игрока. При наличии подходящего намерения сервер не ограничивается генерацией текста, а переходит к tool-oriented обработке: строит детерминированную команду `/give`, `/clear` или `/summon`, возвращая модулю клиента структурированный ответ с типом действия, аргументами и финальной репликой. Такой подход исключает прямое исполнение произвольного текста модели и сохраняет единый контракт взаимодействия между сервером и модом.

Дополнительный программный интерфейс реализован для управления состоянием системы. Эндпоинт `/api/settings/` отвечает за чтение и изменение конфигурации, `/api/rp/models/*` — за выбор доступных моделей, а роуты `/api/challenges/*` — за работу с челленджами поверх телеметрии игрока. Для предметов и сущностей используются локальные каталоги, благодаря которым запросы на выдачу предметов, удаление из инвентаря и призыв мобов сначала проходят стадию нормализации и разрешения в канонический идентификатор, и только затем превращаются в команду для игрового клиента.

С точки зрения программного интерфейса особое значение имеет механизм деградации. Если внешний LLM-сервис недоступен или не настроен, система продолжает обслуживать пользовательский сценарий за счёт rule-based ветви обработки. Это позволяет сохранить работоспособность базового ассистирования, мониторинга и администрирования даже в случае отказа внешнего когнитивного компонента.

3.3 Тестирование интерфейса

Функциональное и интеграционное тестирование

Проверка работоспособности реализованного интерфейса проводилась на двух уровнях: автоматизированном и ручном. Базовая автоматизированная часть основана на модуле `unittest` и выполняется командой `python -m unittest discover -s tests -v`. Этот набор тестов покрывает ключевые механизмы серверного интерфейса: разбор и маршрутизацию tool-запросов из чата, разрешение предметов и сущностей по русским и английским названиям, загрузку и переключение LLM-конфигурации, обновление состояния игрока, работу локальной базы крафтов, логику чел-

ленджей и поведение fallback-ветви при недоступности модели.

Дополнительно для проверки именно пользовательского веб-интерфейса был реализован отдельный браузерный smoke-тест на базе *Selenium*. Тестовый сценарий автоматически поднимает backend через *Uvicorn*, запускает headless-браузер Microsoft Edge и выполняет две реальные пользовательские проверки: загрузку страницы `/dashboard` с переходом в раздел крафтов и работу страницы `/admin/wiki` с поиском рецепта “палка” и визуализацией сетки крафта. Такой подход позволяет подтвердить, что клиентский JavaScript, маршруты FastAPI и серверные API работают согласованно, а не только проходят изолированные unit-проверки.

По итогам комплексного прогона зафиксировано 77 успешно пройденных тестов (рисунок 3.5), включая server-side проверки и два UI smoke-сценария в браузере. (рисунок 3.6).

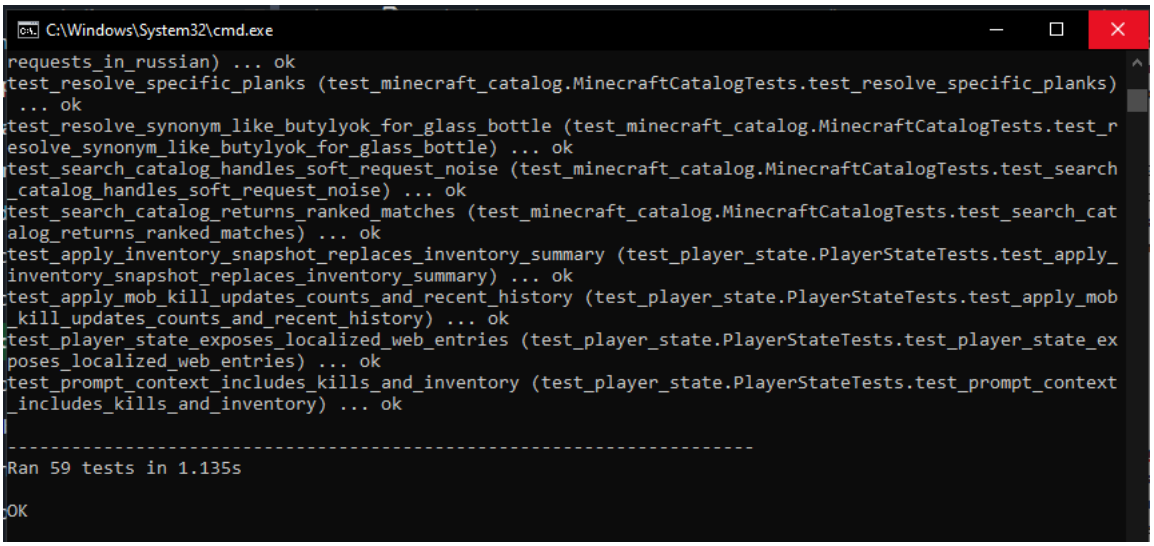


Рисунок 3.5 – Результаты unit-тестов

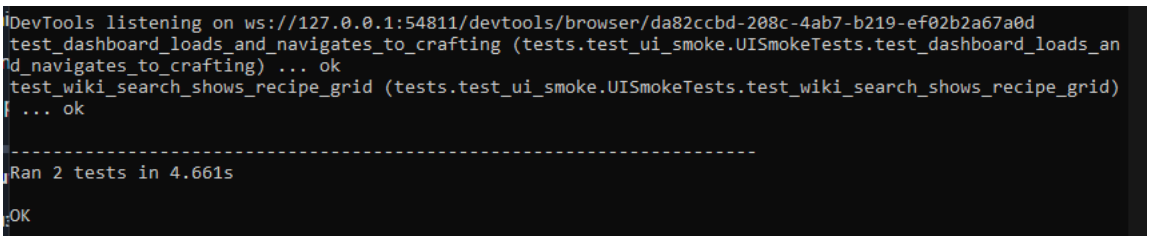


Рисунок 3.6 – Результаты smoke-теста

Интеграционная проверка проводилась также в ручном режиме при совместном запуске FastAPI-сервера и клиентского Fabric-мода. В рамках сценариев тестирования контролировались: появление событий в административной панели, корректность ответов ассистента на игровые сообщения, выполнение команд выдачи предметов и призыва мобов, обновление

данных инвентаря и логирование убийств через серверное событие `AFTER_DEATH`. Отдельно проверялись переключение моделей, сохранение настроек, отображение истории ответов, корректная работа расширенных debug-страниц и загрузка альтернативных web-маршрутов `/dashboard` и `/steam`.

Оценка удобства и качества взаимодействия

Оценка удобства взаимодействия выполнялась методом экспертной эвристической оценки по критериям понятности, непрерывности сценария, наблюдаемости состояния и устойчивости к ошибкам. Реализованный интерфейс удовлетворяет базовым требованиям usability: основное взаимодействие игрока происходит в естественной для среды форме, без перехода в отдельное приложение; административная панель обеспечивает прозрачность работы системы; критические ошибки внешних сервисов не приводят к полной потере функциональности.

Система агрегирует телеметрию, ведёт контекст диалога, различает режимы реакций, детерминированно исполняет игровые действия через tool-layer и поддерживает механику челленджей на базе состояния игрока. При этом объяснимость частично сохраняется за счёт явного поля источника ответа, наблюдаемого потока логов и административных debug-экранов.

3.4 Вывод по разделу реализации

В результате реализации был создан работоспособный прототип интеллектуальной системы ассистирования для Minecraft, включающий серверный контур на FastAPI, клиентский мод на Fabric и веб-панель администратора. Поддерживаются следующие сценарии взаимодействия: автоматический сбор телеметрии, контекстная генерация реплик, диалог игрока с ассистентом через чат, безопасное исполнение игровых команд через tool-слой, мониторинг событий в режиме реального времени и управление конфигурацией системы.

Практическая реализация подтвердила корректность архитектурных решений, принятых на этапе проектирования. Разделение обязанностей между модом, сервером и внешним LLM-провайдером позволило локализовать сложную логику на сервере и сохранить сравнительно простой контракт обмена с клиентом. Использование локальных каталогов, отдельного состояния игрока и JSON-персистентности для настроек и челленджей повысило предсказуемость поведения интерфейса и упростило его сопровождение.

Текущий прототип готов к демонстрации в режиме реальной работы, однако сохраняет потенциал дальнейшего развития. Перспективными направлениями являются расширение числа поддерживаемых инструментальных действий, развитие голосового канала обратной связи, усиление аналитических возможностей административной панели и дальнейшее

углубление контекстной памяти ассистента. Таким образом, разработанная система может рассматриваться как функциональная база для последующего перехода от курсового прототипа к более зрелому интеллектуальному интерфейсу.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была изучена предметная область интеллектуальных интерфейсов для виртуальных сред, исследованы существующие подходы к автоматизации взаимодействия пользователя с игровым миром и сформулированы требования к проектируемой системе ассистирования. Проведённый анализ показал, что традиционные интерфейсные решения либо перегружают пользователя избыточной визуальной информацией, либо не обеспечивают гибкого контекстного взаимодействия в реальном времени.

На этапе проектирования была предложена клиент-серверная архитектура интеллектуальной системы, включающая мод для сбора телеметрии в среде Minecraft, серверную часть на базе FastAPI для агрегации событий и принятия решений, а также веб-интерфейсы для мониторинга, настройки и работы со справочной информацией. Определены основные сценарии взаимодействия между игроком, административным пользователем, игровым клиентом и внешними когнитивными сервисами. Сформулированы требования к надёжности, расширяемости, объяснимости и адаптивности интерфейса.

В результате практической реализации разработан работоспособный прототип интеллектуального ассистента, поддерживающий автоматический сбор игровых событий, генерацию контекстных реплик, обработку пользовательского чата, безопасное выполнение tool-ориентированных команд, работу с системой челленджей, отображение состояния игрока и справочник рецептов крафта. Показано, что веб-контур может сочетать как классические административные экраны, так и более современные формы представления информации: dashboard с оперативной сводкой состояния системы и отдельный Steam-style интерфейс библиотеки крафтов.

Проведено тестирование реализованного интерфейса. Автоматизированная проверка охватила серверную логику и пользовательский веб-контур, включая Selenium smoke-тесты браузерных сценариев. Результаты тестирования подтвердили корректность ключевых пользовательских операций, согласованность клиентской и серверной частей, а также работоспособность обновлённого веб-интерфейса после интеграции изменений из смежной ветки разработки.

Таким образом, задачи, поставленные во введении, в рамках курсового проекта решены: выполнены анализ предметной области и аналогов, спроектирована архитектура интеллектуальной системы, реализованы основные пользовательские и программные интерфейсы, проведено тестирование и подтверждена пригодность прототипа к демонстрации в реальном режиме работы. Ожидаемый эффект применения разработанного решения заключается в снижении когнитивной нагрузки на пользователя, повыше-

нии наглядности состояния системы и создании основы для дальнейшего развития мультимодального интеллектуального ассистента.

В качестве направлений дальнейшего развития предлагаются расширение набора инструментальных действий ассистента, углубление голосового и мультимодального взаимодействия, развитие аналитических и справочных возможностей веб-интерфейса, а также усиление контекстной памяти и качества объяснимости решений, принимаемых интеллектуальной системой.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Nielsen, Jakob. Usability Engineering / Jakob Nielsen. — San Francisco : Morgan Kaufmann, 1993. — 362 P. — URL: https://www.nngroup.com/books/usability-engineering/ (дата обращения: 05.03.2025).

[2] Csikszentmihalyi, Mihaly. Flow: The Psychology of Optimal Experience / Mihaly Csikszentmihalyi. — New York : Harper and Row, 1990. — 303 P. — URL: https://www.harpercollins.com/products/flow-mihaly-csikszentmihalyi (дата обращения: 05.03.2025).

[3] Language Models are Few-Shot Learners / Tom B. Brown, Benjamin Mann, Nick Ryder [et al.] // Advances in Neural Information Processing Systems. — 2020. — Vol. 33. — Pp. 1877–1901. — URL: https://arxiv.org/abs/2005.14165 (дата обращения: 05.03.2025).

[4] Voyager: An Open-Ended Embodied Agent with Large Language Models / Guanzhi Wang, Yuqi Xie, Yunfan Jiang [et al.] // arXiv preprint arXiv:2305.16291. — 2023. — URL: https://arxiv.org/abs/2305.16291 (дата обращения: 05.03.2025).

[5] The Rise and Potential of Large Language Model Based Agents: A Survey / Zhiheng Xi, Wenxiang Chen, Xin Guo [et al.] // arXiv preprint arXiv:2309.07864. — 2023. — URL: https://arxiv.org/abs/2309.07864 (дата обращения: 05.03.2025).

[6] Baritone Developers. Baritone: A pathfinding system for Minecraft. — 2024. — URL: https://github.com/cabaletta/baritone (дата обращения: 05.03.2025). https://github.com/cabaletta/baritone.

[7] Mojang Studios. Minecraft Accessibility Menu and Text-to-Speech. — 2024. — URL: https://help.minecraft.net/hc/en-us/articles/360059444232 (дата обращения: 05.03.2025).

[8] Sweller, John. Cognitive load during problem solving: Effects on learning / John Sweller // Cognitive Science. — 1988. — no. 12. — Pp. 257–285. — URL: https://onlinelibrary.wiley.com/doi/abs/10.1207/s15516709cog1202_4 (дата обращения: 05.03.2025).

[9] Generative Agents: Interactive Simulacra of Human Behavior / Joon Sung Park, Patrick O'Keefe, Carrie Cai [et al.] // Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23). — Association for Computing Machinery, 2023. — Pp. 1–22. — URL: https://arxiv.org/abs/2304.03442 (дата обращения: 05.03.2025).

[10] Fabric Project. Fabric: A lightweight, experimental modding toolchain for Minecraft. — 2024. — URL: https://fabricmc.net/ (дата обращения: 05.03.2025).